

Sprawozdanie: Rozprzestrzenianie Koloru (Dyfuzja na Siatce 2D)

Łukasz Wołoszyn

Jakub Wójcikiewicz

CY4

1. Opis Projektu

Projekt realizuje symulację rozlewania się koloru po dwuwymiarowej siatce. Na początku w kilku punktach siatki rozmieszczone są źródła koloru (czerwony, zielony, niebieski, żółty, magenta, cyjan), które z każdą iteracją rozlewają się na sąsiednie pola. Wartość koloru w każdej komórce jest obliczana jako średnia z jej sąsiadów, co modeluje proces dyfuzji podobny do rozchodzenia się ciepła lub farby na powierzchni.

Algorytm dyfuzji

W każdej iteracji, dla każdej komórki (x, y) obliczana jest nowa wartość jako średnia arytmetyczna wartości jej sąsiadów. Dostępne są dwa szablony:

- 5-punktowy:** centrum + 4 sąsiadów (boki)
- 9-punktowy:** centrum + 8 sąsiadów (w tym przekątne)

Wzór (szablon 5-punktowy):

```
new[y][x] = (old[y][x] + old[y-1][x] + old[y+1][x] + old[y][x-1] + old[y][x+1]) / 5
```

Na brzegach siatki stosowane są warunki brzegowe (komórki poza siatką nie są uwzględniane, dzielnik jest odpowiednio zmniejszony). Po każdej iteracji przywracane są oryginalne wartości w punktach źródłowych.

2. Podział Pracy

Zadanie	Łukasz	Jakub
Implementacja sekwencyjna	✓	
Implementacja OpenMP		✓
Implementacja MPI	✓	
Implementacja CUDA		✓
Wizualizacja GUI (Python)	✓	✓
Testy wydajnościowe	✓	✓

3. Konfiguracja Testowa

Komputer Testowy #1

Parametr	Wartość
Procesor	AMD Ryzen 7 4800H
Rdzenie/Wątki	8 rdzeni / 16 wątków
RAM	16 GB (8 GB przydzielone dla WSL)
System	Ubuntu 24.04 (WSL) pod kontrolą Windows
Karta Graficzna	NVIDIA GeForce RTX 2060 (6 GB GDDR6)

Komputer Testowy #2

Parametr	Wartość
Procesor	Intel i5-12400F
Rdzenie/Wątki	8 rdzeni / 12 wątków
RAM	32 GB (8 GB przydzielone dla WSL)
System	Ubuntu 24.04 (WSL) pod kontrolą Windows
Karta Graficzna	NVIDIA GeForce RTX 4060 (8 GB GDDR6)

4. Objaśnienia Kluczowych Fragmentów Kodu

4.1 Implementacja Sekwencyjna

Bazowa implementacja wykorzystuje podwójne buforowanie (ping-pong). Dwie siatki `gridA` i `gridB` są zamieniane po każdej iteracji, co eliminuje konieczność kopiowania danych.

```
// Podwójne buforowanie - w każdej iteracji czytamy z jednej siatki, pisz
Grid* current = &gridA;
Grid* next = &gridB;
for (int iter = 1; iter <= iterations; iter++) {
    diffuseStep5(*current, *next); // Odczyt z current, zapis do next
    next->applySources(sources);    // Przywrócenie źródeł
    std::swap(current, next);       // Zamiana buforów
}
```

4.2 OpenMP

Zrównoleglenie polega na podziale pętli zewnętrznej (po wierszach) między wątki za pomocą dyrektywy `#pragma omp parallel for`.

```
#pragma omp parallel for schedule(static) num_threads(numThreads)
for (int y = 0; y < H; y++) {
    for (int x = 0; x < W; x++) {
        // Obliczenie dyfuzji dla komórki (x, y)
    }
}
```

4.3 MPI

Siatka dzielona jest na bloki wierszy (dekompozycja 1D). Wymiana halo odbywa się w każdej iteracji za pomocą `MPI_Sendrecv`.

```
// Wymiana z sąsiadem powyżej
MPI_Sendrecv(
    curBuf + N*3, N*3, MPI_FLOAT, prevRank, 0,
    curBuf,      N*3, MPI_FLOAT, prevRank, 1,
    MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

4.4 CUDA

Implementacja wykorzystuje pamięć współdzieloną (shared memory) do minimalizacji dostępu do pamięci globalnej GPU.

```
extern __shared__ float tile[];
int tileW = blockDim.x + 2; // szerokość kafelka z halo

// Ładowanie centralnej części
int gidx = (gy * W + gx) * 3;
int lidx = (ly * tileW + lx) * 3;
tile[lidx] = input[gidx];

__syncthreads(); // Synchronizacja

// Obliczenia na shared memory
float sR = tile[(ly*tileW+lx)*3];
sR += tile[((ly-1)*tileW+lx)*3];
```

5. Instrukcja Obsługi

5.1 Kompilacja i przygotowanie środowiska

```
# 1. Instalacja zależności dla Pythona
pip install -r requirements.txt

# 2. Kompilacja plików wykonywalnych C++/CUDA (najlepiej w WSL2 lub Linux)
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j$(nproc)
cd ..
```

5.2 Uruchomienie aplikacji

```
# Uruchamiając z poziomu Windowsa w folderze projektu:
wsl -e bash -c "python3 gui.py"
# Natywnie z poziomu Linuxa / wewnątrz WSL:
python3 gui.py
```

Z poziomu graficznego interfejsu można konfigurować parametry (rozmiar siatki, szablon, ilość wątków/bloków itp.) i oglądać animację dyfuzji na żywo. Aplikacja automatycznie wywołuje skompilowane wersje i ładuje ich wyniki z powrotem do UI.

6. Testy Wydajnościowe

6.1 Parametry testów

- Rozmiary siatki: 500×500, 1000×1000, 2000×2000
- Iteracje: 1000
- Szablon: 5-punktowy
- OpenMP: 1, 2, 4, 8 wątków
- MPI: 1, 2, 4, 8 procesów
- CUDA: bloki 8×8, 16×16, 32×32

Cały zestaw testów generowany jest automatycznie po naciśnięciu przycisku **Benchmark** w interfejsie graficznym, co powoduje wywołanie w tle skryptu `scripts/run_benchmarks.sh`.

6.2 Wyniki

Tabela dla siatki 500x500

Technologia	Rozmiar	Wątki/Proc./Bloki	Czas [s]	Przyspieszenie	Efektywność
Sekwencyjna	500	1	1.2443	1.00x	100.0%
OpenMP	500	1	1.0681	1.16x	116.0%
OpenMP	500	2	0.5778	2.15x	107.5%
OpenMP	500	4	0.3029	4.11x	102.8%
OpenMP	500	8	0.2567	4.85x	60.6%
MPI	500	1	1.0786	1.15x	115.0%

Technologia	Rozmiar	Wątki/Proc./Bloki	Czas [s]	Przyspieszenie	Efektywność
MPI	500	2	0.5828	2.14x	107.0%
MPI	500	4	0.3494	3.56x	89.0%
MPI	500	8	0.2743	4.54x	56.8%
CUDA	500	8x8	0.0754	16.50x	206.2%
CUDA	500	16x16	0.0663	18.77x	117.3%
CUDA	500	32x32	0.1480	8.41x	26.3%

Tabela dla siatki 1000x1000

Technologia	Rozmiar	Wątki/Proc./Bloki	Czas [s]	Przyspieszenie	Efektywność
Sekwencyjna	1000	1	3.9282	1.00x	100.0%
OpenMP	1000	1	4.0937	0.96x	96.0%
OpenMP	1000	2	2.3296	1.69x	84.5%
OpenMP	1000	4	1.6635	2.36x	59.0%
OpenMP	1000	8	1.4195	2.77x	34.6%
MPI	1000	1	4.1395	0.95x	95.0%
MPI	1000	2	2.3473	1.67x	83.5%
MPI	1000	4	1.6478	2.38x	59.5%
MPI	1000	8	1.4123	2.78x	34.8%
CUDA	1000	8x8	0.2096	18.74x	234.2%
CUDA	1000	16x16	0.1583	24.82x	155.1%
CUDA	1000	32x32	0.1458	26.94x	84.2%

Tabela dla siatki 2000x2000

Technologia	Rozmiar	Wątki/Proc./Bloki	Czas [s]	Przyspieszenie	Efektywność
Sekwencyjna	2000	1	15.7560	1.00x	100.0%
OpenMP	2000	1	14.8360	1.06x	106.0%
OpenMP	2000	2	7.5313	2.09x	104.5%
OpenMP	2000	4	6.7782	2.32x	58.0%

Technologia	Rozmiar	Wątki/Proc./Bloki	Czas [s]	Przyspieszenie	Efektywność
OpenMP	2000	8	5.5870	2.82x	35.3%
MPI	2000	1	14.9570	1.05x	105.0%
MPI	2000	2	7.7040	2.05x	102.5%
MPI	2000	4	6.2920	2.50x	62.5%
MPI	2000	8	5.4114	2.91x	36.4%
CUDA	2000	8x8	0.7647	20.60x	257.5%
CUDA	2000	16x16	0.6052	26.03x	162.7%
CUDA	2000	32x32	0.5261	29.95x	93.6%

6.3 Wykresy

Po przeprowadzeniu testów, wykresy analityczne są renderowane do plików PNG do folderu `results/wykresy/` i można je przeglądać wewnątrz aplikacji GUI w zakładce "Benchmark".

7. Analiza Wyników

7.1 OpenMP

- Przyspieszenie rośnie z liczbą wątków, ale z malejącą efektywnością
- Dla małych siatek overhead tworzenia wątków dominuje
- `schedule(static)` jest optymalny - równomierny rozkład pracy

7.2 MPI

- Komunikacja (halo exchange) stanowi stały narzut w każdej iteracji
- Dekompozycja 1D (wierszowa) minimalizuje objętość komunikacji
- Dla dużych siatek stosunek komunikacji do obliczeń maleje → lepsza skalowalność

7.3 CUDA

- Największe przyspieszenie dzięki masowemu paralelizmowi GPU
- Shared memory eliminuje redundantne odczyty z pamięci globalnej
- Transfer CPU↔GPU to jednorazowy koszt (na początku i na końcu)

8. Wnioski

- **Skalowalność dla małych siatek (500x500):** W przypadku mniejszych rozmiarów danych, obie implementacje wielordzeniowe na CPU (OpenMP i MPI) skalują się bardzo dobrze dochodząc do przyspieszenia rzędu 4.8x przy 8 wątkach. Wersja GPU z blokiem 16x16 daje ogromne przyspieszenie (18.8x), jednakże blok 32x32 jest w tym przypadku paradoksalnie znacznie wolniejszy (8.4x), co wynika ze zbyt małej siatki do optymalnego wypełnienia tak dużych bloków.
- **Skalowalność dla dużych siatek (1000x1000, 2000x2000):** W miarę wzrostu rozmiaru problemu, obie wersje CPU szybciej napotykają tzw. "memory wall" (wąskie gardło pamięci RAM). Przy 8 wątkach ich wydajność "spłaszcza" się w okolicach maksymalnie 2.8x do 2.9x (efektywność drastycznie spada do poziomu ~35%).
- **Zestawienie OpenMP a MPI:** Niezależnie od rozmiaru siatki, OpenMP i MPI na jednej maszynie testowej wykazują identyczne czasy i niemal taką samą skalowalność. Dowodzi to tego, że wymiana pasów granicznych "halo" na tej samej maszynie nie jest dużo wolniejsza niż konwencjonalne synchronizowanie współdzielonej pamięci.
- **Ogromna przewaga CUDA na wielkich siatkach:** Jeśli siatka rośnie, rośnie też przyspieszenie w architekturze GPU. Dla siatki 2000x2000 praca jest w stanie optymalnie obciążyć dostępne multiprocesory przy bloku 32x32, notując najwyższe przyspieszenie w całym eksperymencie, wynoszące blisko **30x** i redukujące czas obliczeń z 15.75s do 0.52s.
- Zbudowany, ujednolicony interfejs graficzny w pełni automatyzuje zbieranie, agregację i reprezentację graficzną danych dla wszystkich tych technologii.